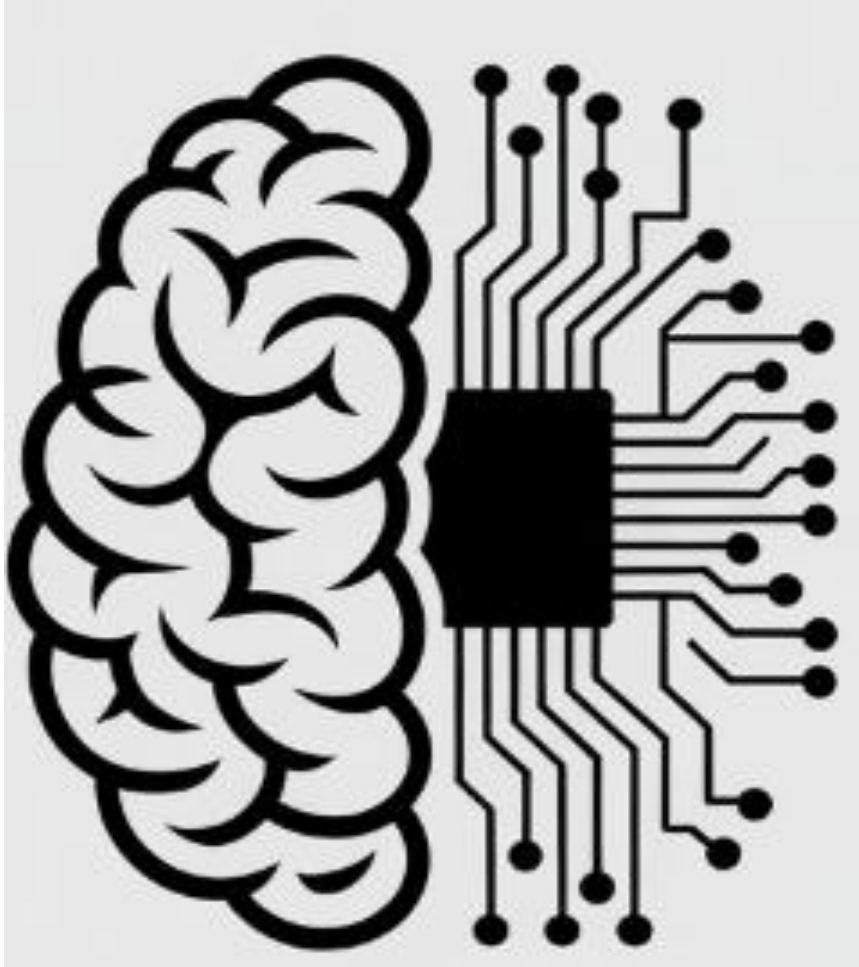
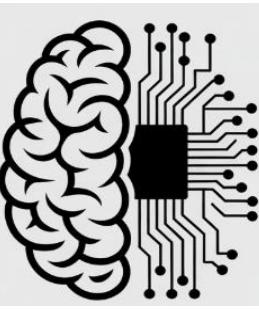




Artificial Intelligence

Dr. Sobia Tariq Javed

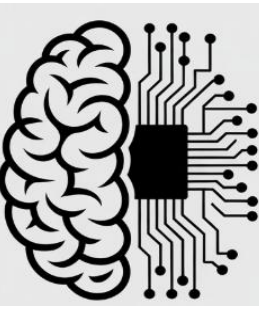
Autonomous Car



- What making it drive?



Autonomous Car

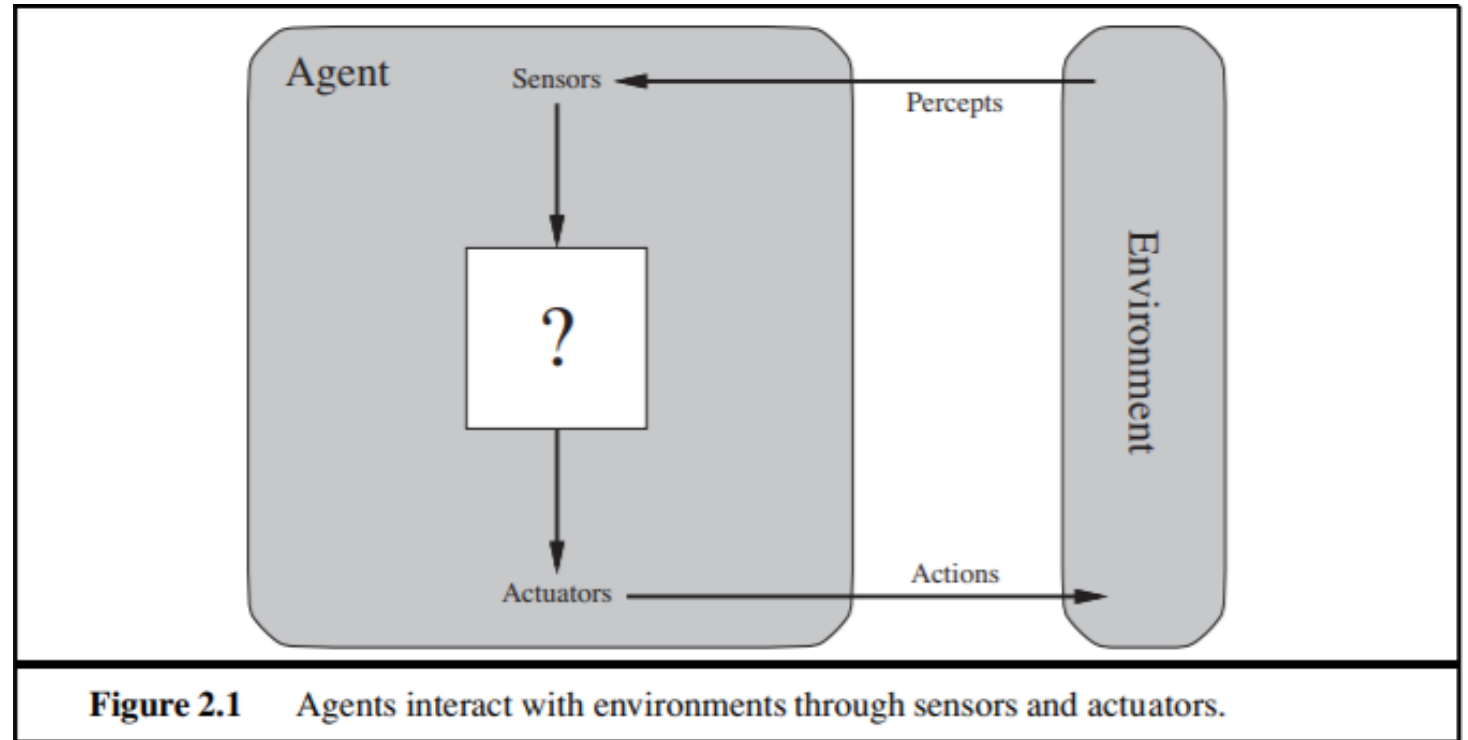


- In a driverless car, the **AI agent** is the autonomous driver that observes the road, makes driving decisions, and controls the vehicle and we need it to safely and autonomously navigate a complex, changing world.



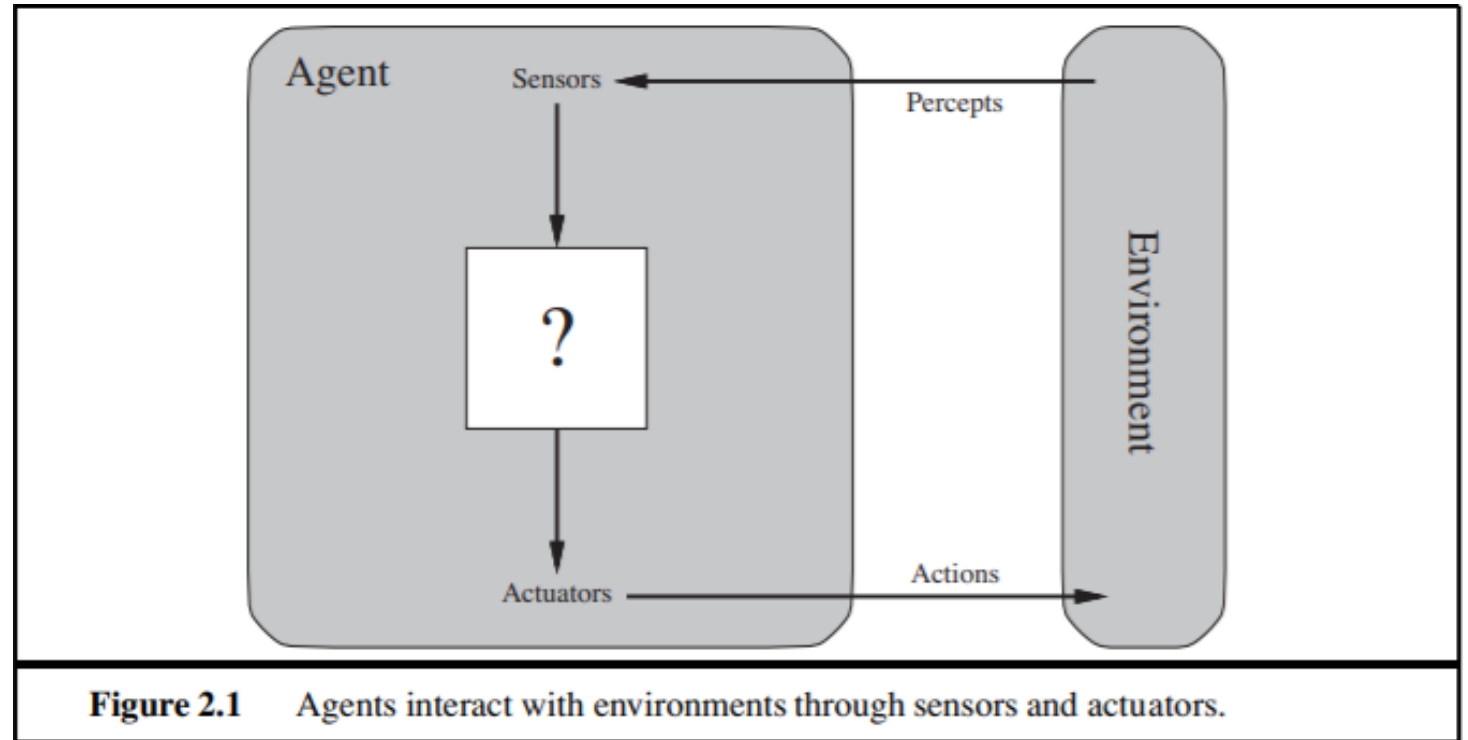
Agents and Environments

- An **agent** is a software or system that is designed to act autonomously and perform specific tasks
- It perceives its **environment** through **sensors** and acts upon that environment through **actuators**
- **Human:**
 - **Sensors:** eyes, ears.
 - **Actuators** (effectors): hands, legs, mouth.

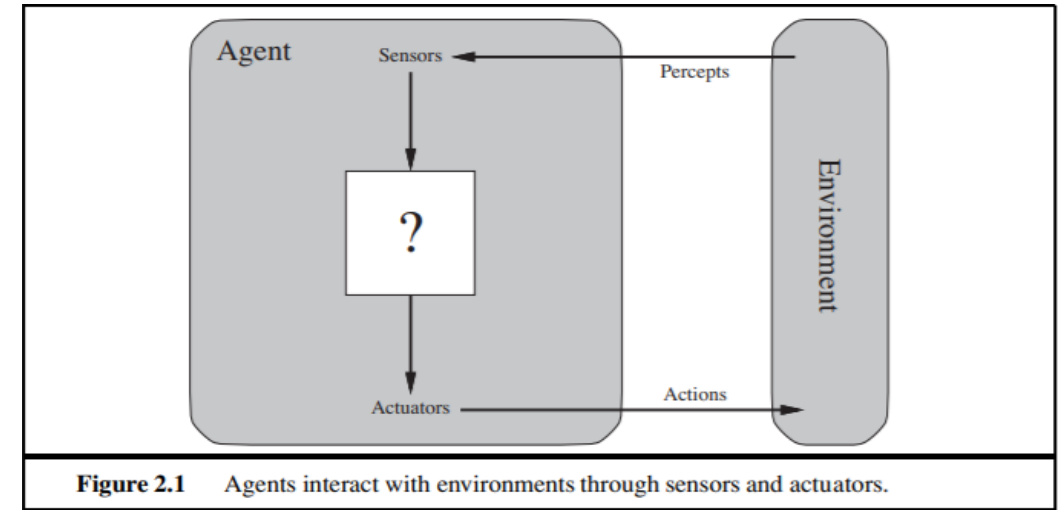


Agents and Environments

- Software Agent
 - **sensory input:** keystrokes, file contents, and network packets
 - **Actuator:** acts on the environment by displaying on the screen, writing files, and sending network packets



Agents and Environments



- **Environment:** The world the agent operates in.
- **Percept:** the sensory input an agent receives from its environment at a given moment.
- **Percept Sequence** = A list of all the inputs (percepts) the agent has gathered over time.
- **Agent Function:** Mapping from an agent's percept or percept sequence to action.
- **Agent Program:** A computational procedure that implements the Agent Function and allows an agent to interact with its environment.
- **Performance Measure:** A metric used to evaluate the effect of action(s).

Rational Agent

- **A rational agent is one that does the right thing**
- **Rationality:**
 - The performance measure that defines criterion of success.
 - The agent prior knowledge of the environment.
 - The actions that agent can perform.

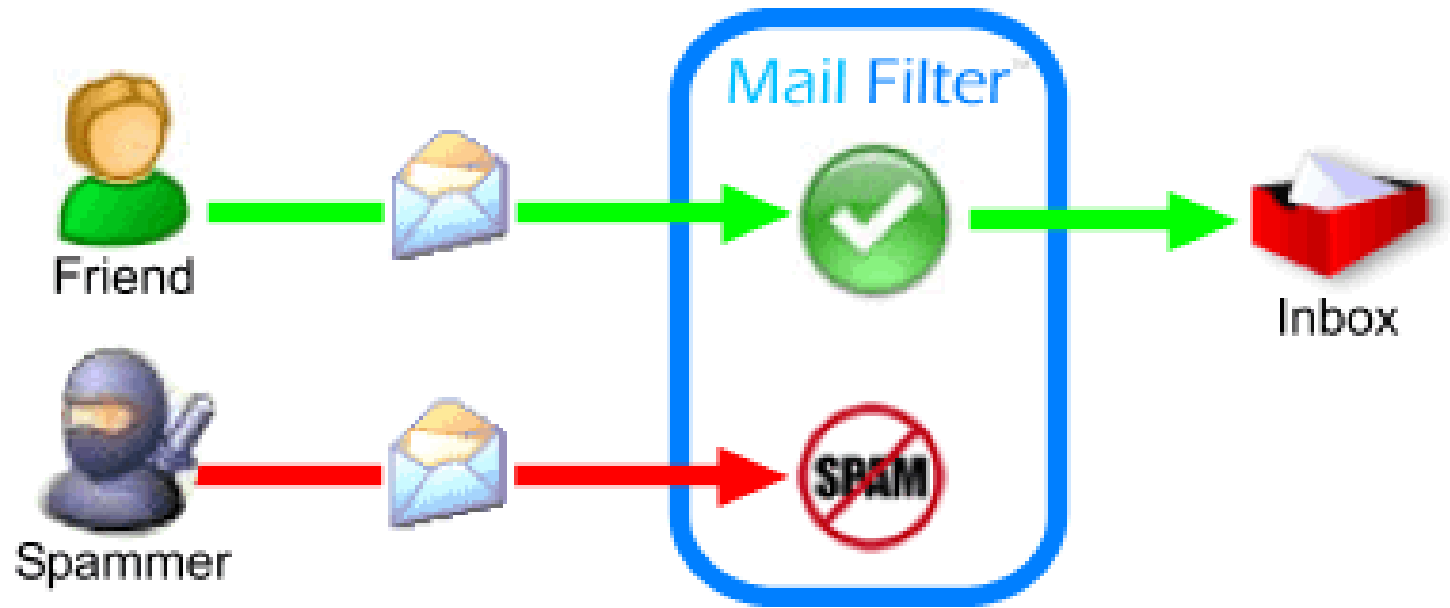
DEFINITION OF A
RATIONAL AGENT



This leads to a **definition of a rational agent**:

For each possible percept sequence, a rational agent should select an action that is expected to maximize its performance measure, given the evidence provided by the percept sequence and whatever built-in knowledge the agent has.

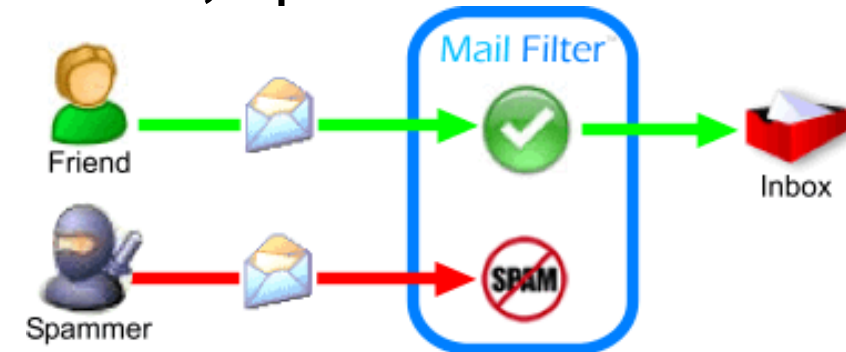
Mail Filter System (Spam Filter)



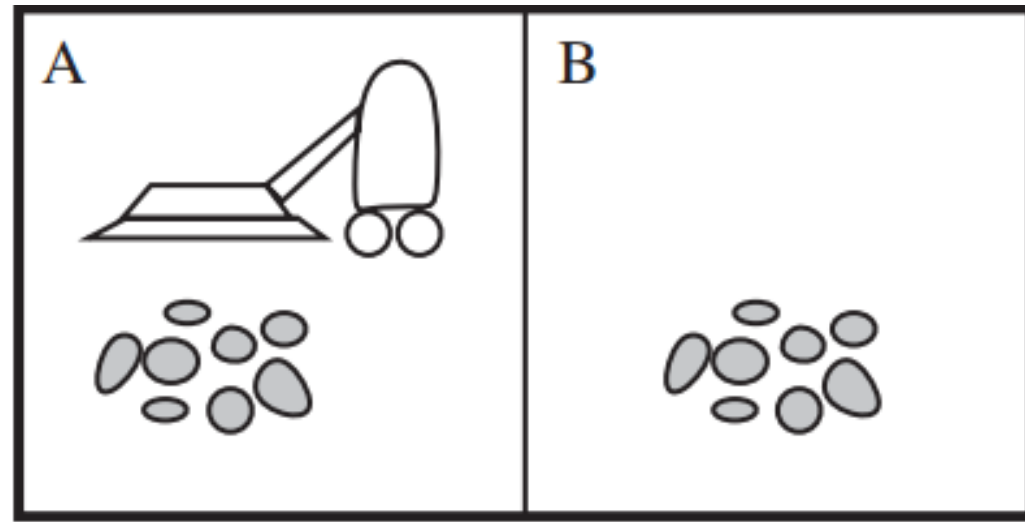
- It is an automated tool designed to analyze incoming emails and classify them as either spam (unwanted or unsolicited) or legitimate (wanted) emails.
- Its primary goal is to keep your inbox free from unwanted content while allowing valid emails to be delivered without issue.

Mail Filter System (Spam Filter)

- **Agent:** Email Spam Filter
- **Environment:** Inbox
- **Sensor:** Content extractor, metadata extractor.
- **Percept:** Email content, sender info.
- **Actuator:** Move to spam/inbox.
- **Agent Function:** Classify as spam/non-spam.
- **Performance Measure:** Accuracy, precision, recall, speed.



A vacuum cleaner world with just two locations



- This particular world has just two locations: **squares A and B.**
- The vacuum agent perceives which square it is in and whether there is **dirt in the square.**
- It can choose to **move left, move right, suck up the dirt,** or do nothing.
- One very simple agent function is the following: **if the current square is dirty, then suck; otherwise, move to the other square.**

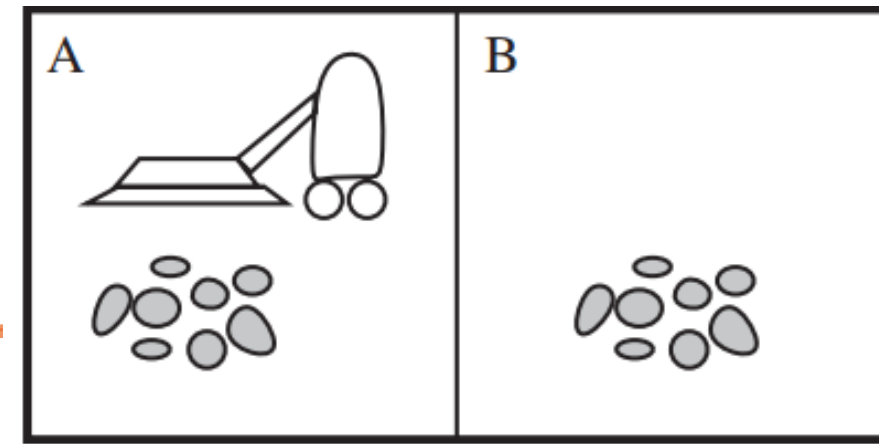
Vacuum Cleaner

Percept sequence	Action
<i>[A, Clean]</i>	<i>Right</i>
<i>[A, Dirty]</i>	<i>Suck</i>
<i>[B, Clean]</i>	<i>Left</i>
<i>[B, Dirty]</i>	<i>Suck</i>
<i>[A, Clean], [A, Clean]</i>	<i>Right</i>
<i>[A, Clean], [A, Dirty]</i>	<i>Suck</i>
<i>⋮</i>	<i>⋮</i>
<i>[A, Clean], [A, Clean], [A, Clean]</i>	<i>Right</i>
<i>[A, Clean], [A, Clean], [A, Dirty]</i>	<i>Suck</i>
<i>⋮</i>	<i>⋮</i>

Figure 2.3 Partial tabulation of a simple agent function for the vacuum-cleaner world shown in Figure 2.2.

Vacuum Cleaner

- **Agent:** Vacuum Cleaner
- **Environment:** Area A and B
- **Sensor:** Camera
- **Percept:** Area Clean or not
- **Actuator:** Sucker, controller to move
- **Agent Function:** Move left, Move right. Start Cleaning
- **Performance Measure:** Coverage, Efficiency, dirt removed



PEAS

- **P**erformance measure
- **E**nvironment
- **A**ctuators
- **S**ensors

Automated Taxi

- **Performance Measure:** Safe, fast, legal, comfortable trip, maximize profits.
- **Environment:** Roads, other traffic, pedestrians, customers.
- **Actuators:** Steering, accelerator, brake, signal, horn, display.
- **Sensor:** Cameras, sonar, speedometer, GPS, odometer, accelerometer, engine sensors, keyboard



Part-picking Robot

- **Performance Measure:** Percentage of parts in correct bins.
- **Environment:** Conveyor belt with parts; bins.
- **Actuators:** Jointed arm and hand.
- **Sensor:** Camera, joint angle sensors.



Properties of Task Environments

1. Fully Observable vs Partially Observable

- **Fully Observable:**

The agent can perceive the complete state of the environment at each time step.

- Example: Two-location vacuum cleaner world (agent knows its location and cleanliness of both rooms).

- **Partially Observable:**

The agent has incomplete or noisy information about the environment.

- Example: Autonomous driving (cannot see around corners or through fog).

2. Deterministic vs. stochastic / strategic

- **Deterministic:**

- The next state is completely determined by the current state and the agent's action.
- Example: Chess (each move has a predictable outcome).
- Two-location vacuum cleaner world

- **Stochastic:**

- Actions may have uncertain outcomes. It involves a certain degree of randomness or probability.
- Example: Robot navigation where sensors or actuators may fail.
- Weather prediction systems

2. Deterministic vs. stochastic / strategic

- **Strategic systems** refers to decision making process where the decision maker take into account the potential actions of other decision makers and their potential reactions to the decision maker's own actions.
- Example
 - Chess (considering opponent moves)
 - Poker
 - Autonomous driving in traffic with other drivers

3. Episodic vs Sequential

- **Episodic:**

- Each action is independent of previous actions.
- Example: Image classification (classifying one image does not affect the next).

- **Sequential:**

- Current actions affect future states and rewards.
- Example: Vacuum cleaner world (cleaning or moving affects future actions).

4. Static vs Dynamic

- **Static:**

- The environment does not change while the agent is deciding.
- Example: Crossword puzzle.

- **Dynamic:**

- The environment changes over time, even without agent actions.
- Example: Stock market, autonomous driving traffic.

- **Semi dynamic:**

- If the environment itself does not change with the passage of time but the agent's performance score does.

4. Static vs Dynamic

- **Semi dynamic:**

- If the environment itself does not change with the passage of time but the agent's performance score does.
- Example
 - Chess with a clock
 - Online exam with a time limit

5. Discrete vs Continuous

- **Discrete:**

- Finite number of states, actions, and percepts.
- Example: Chess (finite board positions and moves).

- **Continuous:**

- Infinite or continuous states or actions.
- Example:
- Robotic arm movement (continuous angles and positions).
- Taxi-driving is continuous-state.

6. Single-Agent vs Multi-Agent

- **Single-Agent:**

- Only one agent acts in the environment.
- Example: Vacuum cleaner agent.

- **Multi-Agent:**

- Multiple agents interact, cooperatively or competitively.
- Example: Chess (two competing agents), traffic systems.

7. Known vs Unknown

- **Known:**

- The agent knows the rules and outcomes of actions.
- Example: Board games like chess.

- **Unknown:**

- The agent must learn the environment dynamics.
- Example: Robot exploring an unknown area.

Environment Properties



Environment	Observable	Deterministic	Episodic	Static	Discrete	Agents
Chess with a clock						
Chess without a clock						

Environment Properties



Environment	Observable	Deterministic	Episodic	Static	Discrete	Agents
Chess with a clock	Fully	Deterministic Strategic	Sequential	Semi	Discrete	Multi
Chess without a clock						

Environment Properties



Environment	Observable	Deterministic	Episodic	Static	Discrete	Agents
Chess with a clock	Fully	Deterministic Strategic	Sequential	Semi	Discrete	Multi
Chess without a clock	Fully	Deterministic Strategic	Sequential	Static	Discrete	Multi

Environment Properties



Environment	Observable	Deterministic	Episodic	Static	Discrete	Agents
Chess with a clock	Fully	Deterministic Strategic	Sequential	Semi	Discrete	Multi
Chess without a clock	Fully	Deterministic Strategic	Sequential	Static	Discrete	Multi
Poker						

Environment Properties



Environment	Observable	Deterministic	Episodic	Static	Discrete	Agents
Chess with a clock	Fully	Deterministic Strategic	Sequential	Semi	Discrete	Multi
Chess without a clock	Fully	Deterministic Strategic	Sequential	Static	Discrete	Multi
Poker	Partially	Deterministic Strategic	Sequential	Static	Discrete	Multi

Environment Properties



Environment	Observable	Deterministic	Episodic	Static	Discrete	Agents
Chess with a clock	Fully	Deterministic Strategic	Sequential	Semi	Discrete	Multi
Chess without a clock	Fully	Deterministic Strategic	Sequential	Static	Discrete	Multi
Poker	Partially	Deterministic Strategic	Sequential	Static	Discrete	Multi
Ludo						

Environment Properties



Environment	Observable	Deterministic	Episodic	Static	Discrete	Agents
Chess with a clock	Fully	Deterministic Strategic	Sequential	Semi	Discrete	Multi
Chess without a clock	Fully	Deterministic Strategic	Sequential	Static	Discrete	Multi
Poker	Partially	Deterministic Strategic	Sequential	Static	Discrete	Multi
Ludo	Fully	Stochastic	Sequential	Static	Discrete	Multi

Environment Properties



Environment	Observable	Deterministic	Episodic	Static	Discrete	Agents
Chess with a clock	Fully	Deterministic Strategic	Sequential	Semi	Discrete	Multi
Chess without a clock	Fully	Deterministic Strategic	Sequential	Static	Discrete	Multi
Poker	Partially	Deterministic Strategic	Sequential	Static	Discrete	Multi
Ludo	Fully	Stochastic	Sequential	Static	Discrete	Multi
Taxi driving	Partial	Stochastic	Sequential	dynamic	Continuous	Multi
Medical diagnosis	Partial	Stochastic	Episodic	Static	Discrete	Single

Structure of Agents

- **Kinds of agents**
 - **Simple reflex agents**
 - **Model based reflex agents**
 - **Goal-based agents**
 - **Utility-based agents**
 - **Learning agents**

1. Simple Reflex Agent

- Acts only on the current percept
- Uses condition–action rules
- No memory of past states
- Example: **Vacuum cleaner** that sucks when dirt is detected
- If the traffic light is red → **Car stops**
- If the front sensor detects an obstacle → **Car applies brake**

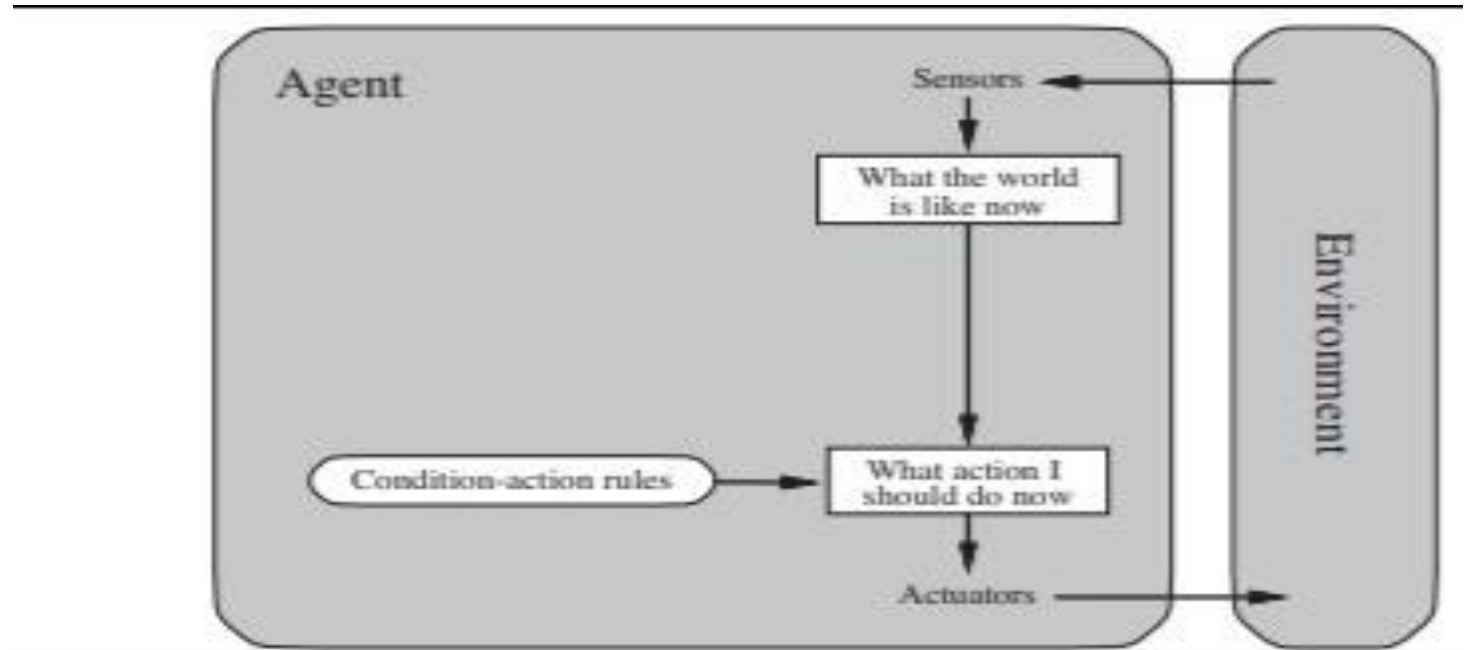


Figure 2.9 Schematic diagram of a simple reflex agent.

2. Model-Based Reflex Agent

- Maintains an internal state (model) of the environment
- Handles partially observable environments
- Example: Vacuum agent remembering which rooms are clean

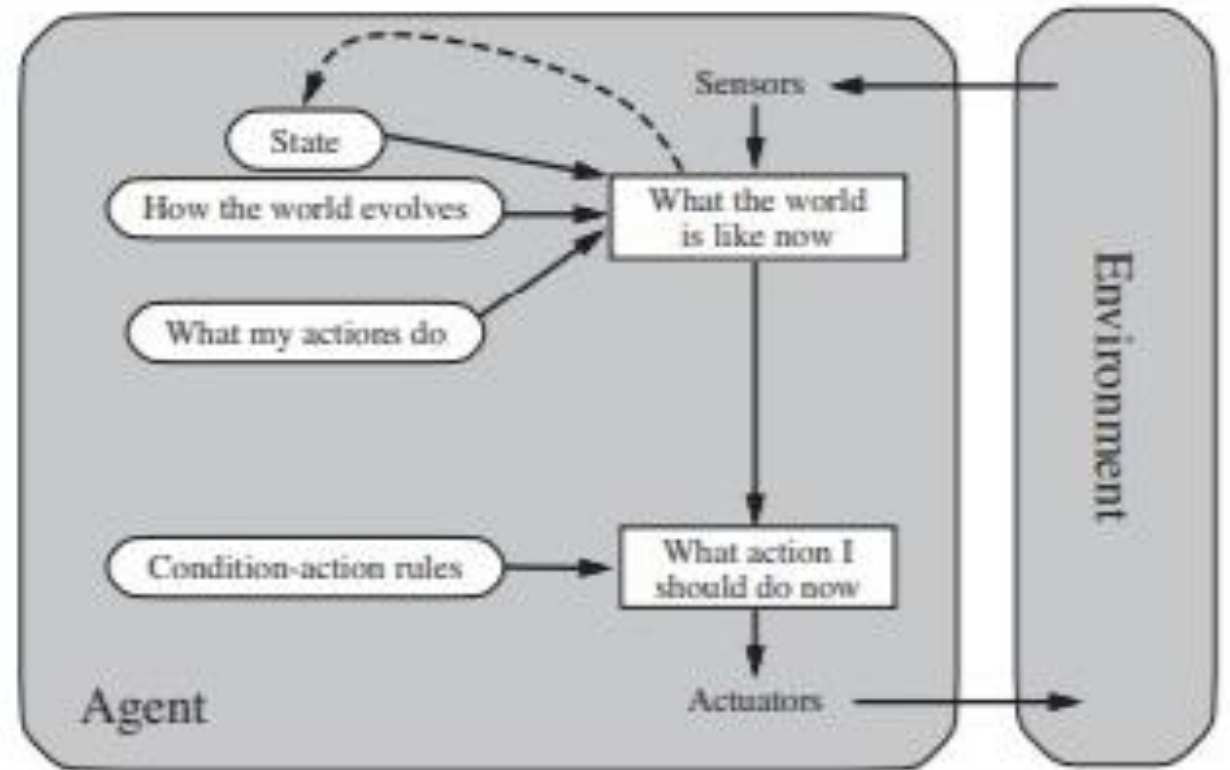


Figure 2.11 A model-based reflex agent.

2. Model-Based Reflex Agent

- **A model-based reflex agent uses past information to understand situations that cannot be observed directly at the moment.**
 - In a **braking problem**, the agent does not need much memory.
 - It only remembers the previous camera image to check whether the red brake lights of the car ahead turn on or off together.
- For **lane changing**, the agent needs more memory.
- It must remember the positions of nearby cars, even when they are not visible at the moment.

3. Goal-Based Agent

- Chooses actions to achieve specific goals
- Uses search and planning
- Example: **GPS navigation system**. For example, at a road junction, the **car** can turn left, turn right, or go straight on. The correct decision depends on where the taxi is trying to get to.

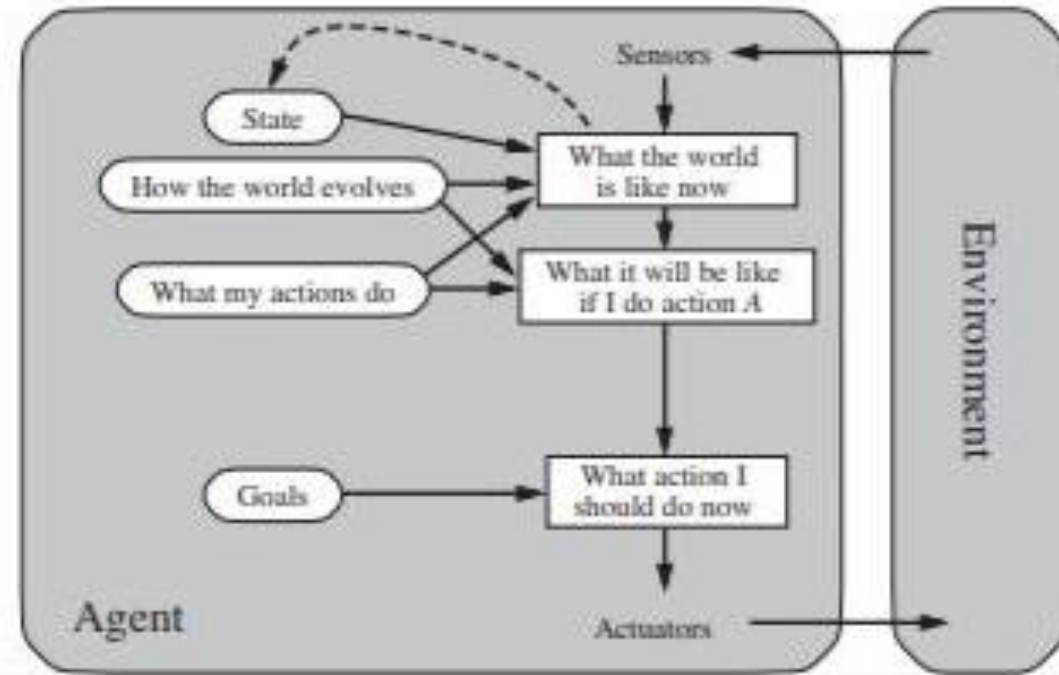


Figure 2.13 A model-based, goal-based agent. It keeps track of the world state as well as a set of goals it is trying to achieve, and chooses an action that will (eventually) lead to the achievement of its goals.

4. Utility-Based Agent

- Selects actions that maximize a utility function
- Handles trade-offs and uncertainty
- Example: **Self-driving car** balancing safety, comfort, fuel efficiency and speed.
- May choose a slightly longer route if it is safer and smoother

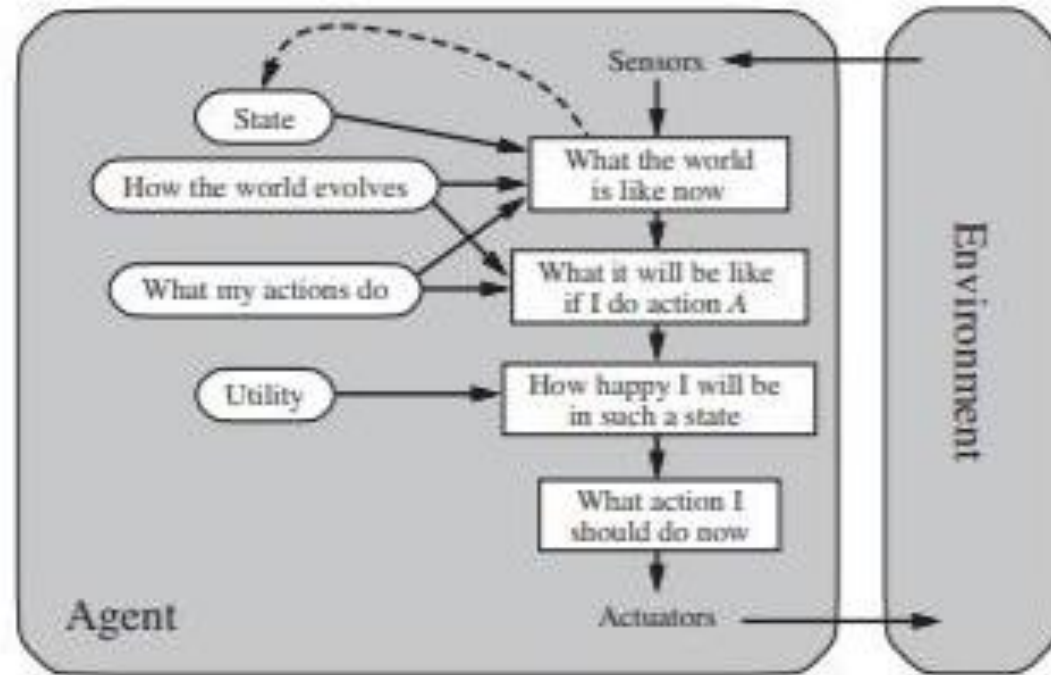


Figure 2.14 A model-based, utility-based agent. It uses a model of the world, along with a utility function that measures its preferences among states of the world. Then it chooses the action that leads to the best expected utility, where expected utility is computed by averaging over all possible outcome states, weighted by the probability of the outcome.

5. Learning Agent

- Improves performance through experience
- Adapts to environment changes
- Example: Recommendation systems, game-playing AI.
- Self driving car learns driving patterns from past trips.
- Improves braking and steering behavior over time using feedback

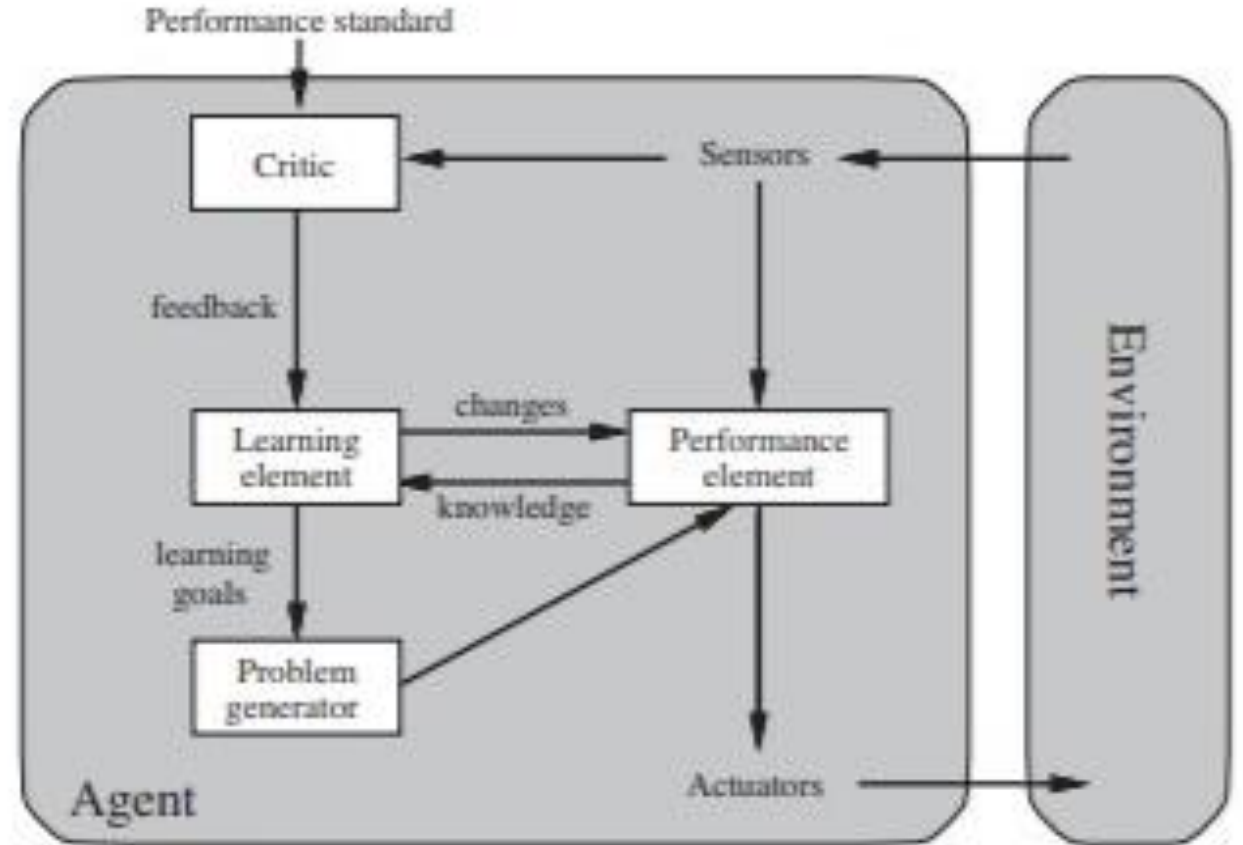


Figure 2.15 A general learning agent.

References

- **CH 2- Intelligent Agent (Section 2.1 - 2.4)**